

Codealpha code review task 3

Project Reviewed: [hack4impact/flask-base](https://github.com/hack4impact/flask-base)

CodeAlpha Cybersecurity Internship — Task 3

Repository: github.com/hack4impact/flask-base

1. Overview

This report documents a secure coding review of flask-base, an open-source Flask application template maintained by hack4impact. The project includes user authentication, role-based permissions, database models (Flask-SQLAlchemy), and form handling (Flask-WTF), making it a representative real-world codebase for security analysis. The review combines automated static analysis (Bandit) with manual inspection of the flagged code.

2. Methodology

Tool used: Bandit (static analysis for Python)

Command: `bandit -r . -x ./venv,./tests`

The scan covered the full project source tree, excluding virtual environment and test directories. Each flagged issue was then manually inspected in context to assess real-world exploitability and impact.

3. Scan Summary

Metric	Result
Total lines of code scanned	1,116
Total issues found	8
Severity: High	2
Severity: Medium	1
Severity: Low	5

4. Detailed Findings

4.1 Command Injection Risk via shell=True (High Severity)

Location: manage.py, line 117

CWE: CWE-78 (OS Command Injection)

The code calls `subprocess.call(yapf, shell=True)`. Running a subprocess with `shell=True` passes the command through the system shell, which interprets special characters (`;`, `&&`, `|`, etc.). If the `yapf` variable is ever influenced by external or untrusted input, this pattern allows arbitrary command execution on the host system.

Recommendation: Avoid `shell=True`. Pass the command as a list of arguments (e.g. `subprocess.call(["yapf", "path/to/file"])`) so the shell is not invoked, eliminating the injection vector.

4.2 Binding to All Network Interfaces (Medium Severity)

Location: manage.py, line 25

CWE: CWE-605 (Multiple Binds to the Same Port)

The development server is configured with `host="0.0.0.0"`, which binds it to every network interface on the machine rather than just localhost. If this configuration is carried into a staging or production-like environment, the application becomes reachable from external networks unintentionally.

Recommendation: Default to `host="127.0.0.1"` for local development, and only bind to `0.0.0.0` deliberately for containerized or intentionally externally-facing deployments, controlled via environment configuration rather than hardcoding.

4.3 Use of assert for Security-Critical Checks (Low Severity)

Location: config.py, line 108

CWE: CWE-703 (Improper Check for Unusual Conditions)

The code uses `assert os.environ.get('SECRET_KEY'), "SECRET_KEY IS NOT SET!"` to enforce that a secret key is configured. Python strips assert statements when run with optimizations enabled (`python -O`), meaning this safeguard can silently disappear in certain production configurations, allowing the app to start without a secret key.

Recommendation: Replace the assert with an explicit check that raises an exception (e.g. `raise RuntimeError(...)`) if `SECRET_KEY` is missing, ensuring the safeguard cannot be optimized away.

4.4 Subprocess Module Usage (Low Severity)

Location: `manage.py`, line 3

The subprocess module is imported and used elsewhere in the same file (see Finding 4.1). Bandit flags the import itself as a marker of increased attack surface, since any use of subprocess warrants careful review of how commands are constructed.

Recommendation: No action needed beyond addressing the specific unsafe call identified in Finding 4.1; the import itself is not inherently unsafe.

5. Conclusion

The flask-base project, while a widely used and generally well-structured Flask template, contains a genuine high-severity command injection risk (`shell=True` usage) and a medium-severity network exposure issue (binding to all interfaces), alongside minor concerns around the robustness of a configuration safeguard. None of these findings are theoretical: each maps to a well-documented CWE category and a concrete, fixable code change. Addressing the `shell=True` usage in `manage.py` should be the top priority, as it represents the most direct path to system compromise.

6. Tools Used

- Bandit (static analysis for Python)
- Manual source code inspection of flagged findings